List Practice Questions:

Q1. Create an ArrayList of integers.

Write a method filterList(ArrayList<Integer> list) that returns a new ArrayList containing only the numbers greater than 50.

```
import java.util.ArrayList;

public class Q1_ArrayList {
   public static ArrayList<Integer> filterList(ArrayList<Integer> list) {
      ArrayList<Integer> filtered = new ArrayList<>();
      for (int num : list) {
         if (num > 50) {
            filtered.add(num);
         }
      }
      return filtered;
   }
   public static void main(String[] args) {
      ArrayList<Integer> data = new ArrayList<>();
      data.add(45);
      data.add(99);
      data.add(10);
      data.add(65);
      ArrayList<Integer> result = filterList(data);
      System.out.println("Original: " + data);
      System.out.println("Filtered: " + result); // Output: [99, 65]
   }
}
```

Q2. Implement a function processOrders(LinkedList<String> orders) that simulates processing a queue: add "OrderX" to the end and then remove the element from the front, printing both the added and removed elements.

```
import java.util.LinkedList;

public class Q4_LinkedList {

    public static void processOrders(LinkedList<String> orders) {

        String newOrder = "Order_" + (orders.size() + 1);


        // Add to the end (Queue behavior: Enqueue) - O(1)

        orders.offerLast(newOrder);

        System.out.println("Added: " + newOrder);


        // Remove from the front (Queue behavior: Dequeue) - O(1)

        String processedOrder = orders.pollFirst();

        System.out.println("Processed: " + processedOrder);

    }


    public static void main(String[] args) {

        LinkedList<String> orderQueue = new LinkedList<>();

        orderQueue.add("InitialOrder");


        processOrders(orderQueue);

        System.out.println("Current Queue: " + orderQueue); // Should contain the new order

    }

}
```

Q3. Given an ArrayList of names, use an Iterator to safely remove all names that have a length of 5. Print the list before and after removal.

```
import java.util.ArrayList;

import java.util.Iterator;

import java.util.List;


public class Q5_SafeRemoval {

   public static void safeRemove(List<String> names) {

      // Must use the Iterator for safe modification

      Iterator<String> it = names.iterator();


      while (it.hasNext()) {

         String name = it.next();

         if (name.length() == 5) {

            it.remove(); // The safe way to modify the list during iteration

         }

      }

   }


   public static void main(String[] args) {

      List<String> names = new ArrayList<>();

      names.add("Alice");

      names.add("Julia"); // Length 5

      names.add("Bob");

      names.add("Smith"); // Length 5

      names.add("Charlie");


      System.out.println("Before Removal: " + names);

      safeRemove(names);

      System.out.println("After Removal: " + names); // Output: [Alice, Bob, Charlie]

}   }
```

Q4. You need to store 10^6 records. You perform 90% lookups by index and 10% insertions/deletions at the end. Choose and justify the better List implementation (ArrayList or LinkedList). Then, demonstrate a single $\text{O}(1)$ insertion at the end using the chosen list.

```
import java.util.ArrayList;

import java.util.List;

public class Q10_ListChoice {

   public static void main(String[] args) {

      // Chosen List for O(1) index access and O(1) end insertion

      List<String> records = new ArrayList<>();

           // Pre-populate with a large number of dummy records

      // (to simulate the high-capacity scenario)

      for (int i = 0; i < 1000000; i++) {

         records.add("Record-" + i);

      }

      // Demonstrate O(1) insertion at the end (appending)

      long startTime = System.nanoTime();

      records.add("NEW_FINAL_RECORD"); // O(1) operation

      long endTime = System.nanoTime();

           System.out.println("Chosen List: ArrayList (Best for O(1) index lookups)");

      System.out.println("Time for O(1) Insertion (Append): " + (endTime - startTime) + "
nanoseconds");

      System.out.println("Total Size: " + records.size());

           // Demonstrate O(1) Lookup:

      System.out.println("O(1) Lookup (get(500000)): " + records.get(500000));

   }

}
```

Q. Write a Java program to store five student names in an ArrayList and print them.

```
import java.util.*;
public class StudentList {
   public static void main(String[] args) {
      ArrayList<String> students = new ArrayList<>();
      students.add("James");
      students.add("Aman");
      students.add("Priya");
      students.add("Rohan");
      students.add("Neha");

      for(String name : students) {
         System.out.println(name);
      }
   }
}
```

Q. Find the Largest Number in an ArrayList

```
import java.util.*;

public class LargestNumber {
   public static void main(String[] args) {
      ArrayList<Integer> numbers = new ArrayList<>(Arrays.asList(12, 45, 67, 23, 89, 34));
      int max = Collections.max(numbers);
      System.out.println("Largest number: " + max);
   }
}
```

Q. Merge two ArrayLists and remove duplicate values.

```
import java.util.*;
public class MergeLists {
   public static void main(String[] args) {
      ArrayList<String> list1 = new ArrayList<>(Arrays.asList("A", "B", "C"));
      ArrayList<String> list2 = new ArrayList<>(Arrays.asList("B", "C", "D", "E"));
      list1.addAll(list2);
      HashSet<String> merged = new HashSet<>(list1);
      System.out.println("Merged List without Duplicates: " + merged);
   }
}
```

Question: Iterate through an ArrayList of integers and print only even numbers.

Solution:

```
ArrayList<Integer> nums = new ArrayList<>(List.of(1, 2, 3, 4, 5, 6));
for (int num : nums) {
   if (num % 2 == 0) {
      System.out.println(num);
   }
}
```